Modelling interactions and behaviour

Adapted from:

Timothy Lethbridge and Robert Laganiere,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 8)

8.1 Interaction Diagrams

Interaction diagrams are used to model dynamic aspects of a software system

- They help you to visualize how the system runs.
- An interaction diagram is often built from a use case and a class diagram.

  —An interaction model shows a set of actors and objects interacting by exchanging messages.

Interactions and messages

- Interaction diagrams show how a set of actors and objects communicate with each other to perform:
  - —The steps of a use case, or
  - —The steps of some other piece of functionality.
- The set of steps, taken together, is called an *interaction*.
- Interaction diagrams can show various types of *messages* exchanged between objects, including:
  - » Simple procedure calls,
  - » Commands issued by an actor through the user interface,
  - » Messages exchanged over a network.
- One of the main objectives of drawing interaction diagrams is to better understand the sequence of messages.

Elements found in interaction diagrams

- Instances of classes (objects)
  - —Shown as boxes with the class and object identifier underlined
- Actors
  - —Shown using the same stick-person symbol as in use case diagrams
- Messages
  - —Shown as arrows from actor to object, or from object to object

Creating interaction diagrams

- Since you need to know the actors and objects involved in the interaction, you should normally develop a class diagram and a use case model before starting to create an interaction diagram.
- Two kinds of diagrams are used to show interactions:
  - —*Sequence diagrams*
  - —*Communication diagrams*

Sequence diagrams

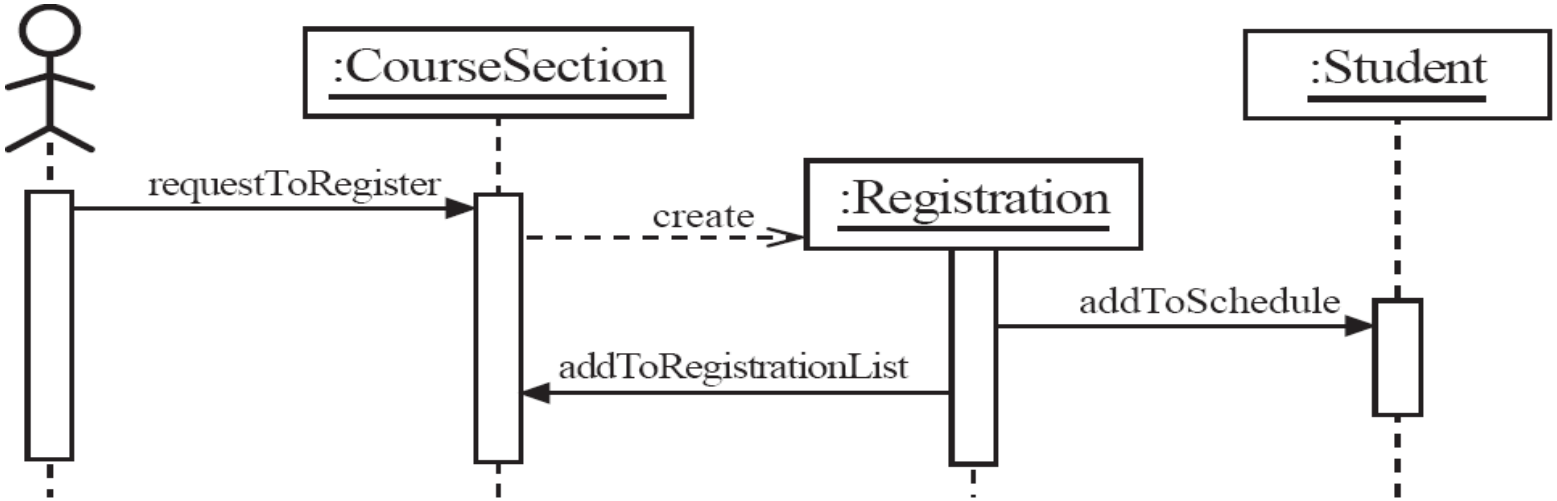
A *sequence diagram* shows the sequence of messages exchanged by the set of objects performing a certain task.

- The objects are arranged horizontally across the diagram.
- An actor that initiates the interaction is often shown on the left.
- The vertical dimension represents time; the top of the diagram is the starting point and time progresses downwards towards the bottom of the diagram.
- A vertical dashed line, called a *lifeline*, is attached to each object or actor.
- The lifeline becomes a box, called an *activation box,* during the period of time (called *live activation* period) when the object is performing computations.
- A *message* is represented as an arrow between activation boxes of the sender and receiver.
  - —You give each message a label; it can optionally have an argument list and a response (return value). The complete syntax is as follows:

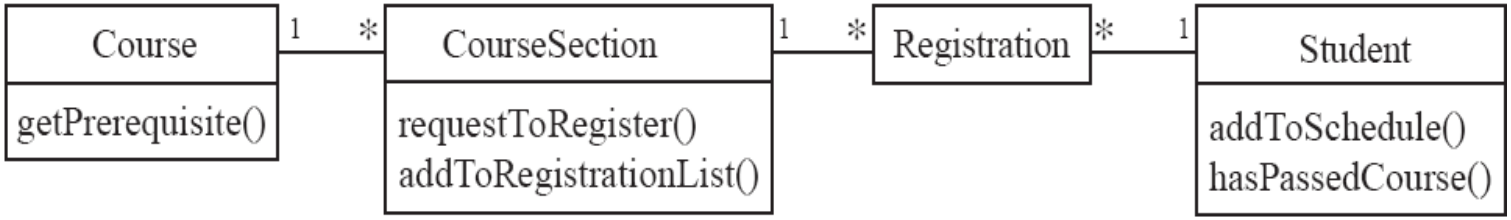
    response := message(arg,…)
  - —A *creation message* is shown using a dashed line with the label ‘create’.

Sequence diagrams – an example

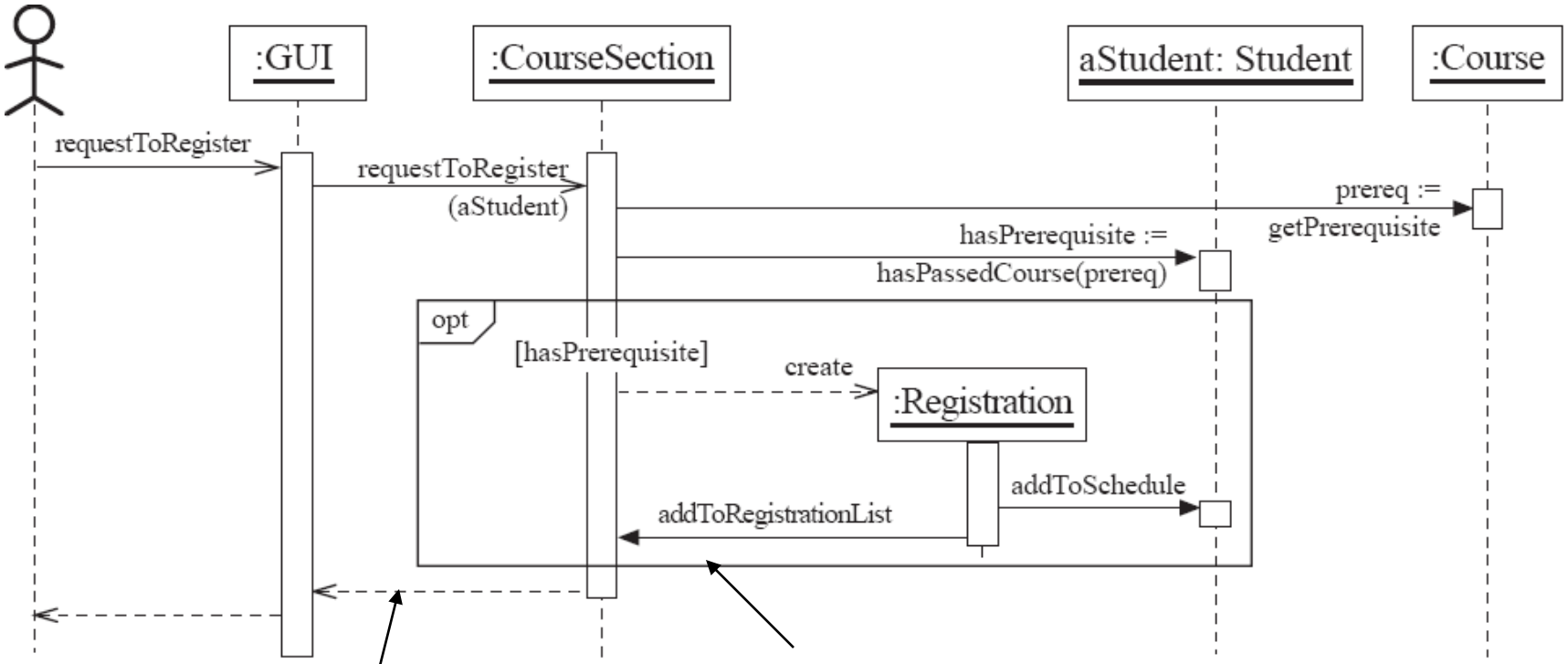
Sequence diagram showing a student registration process:



- The labels on the messages in the sequence diagram correspond to operations in the corresponding class diagram (given below).
- The reception of a message by an object causes one of its methods to be run.
- Sequence diagrams are useful for identifying the operations that have to be included in each class.



Sequence diagrams – same example, more details



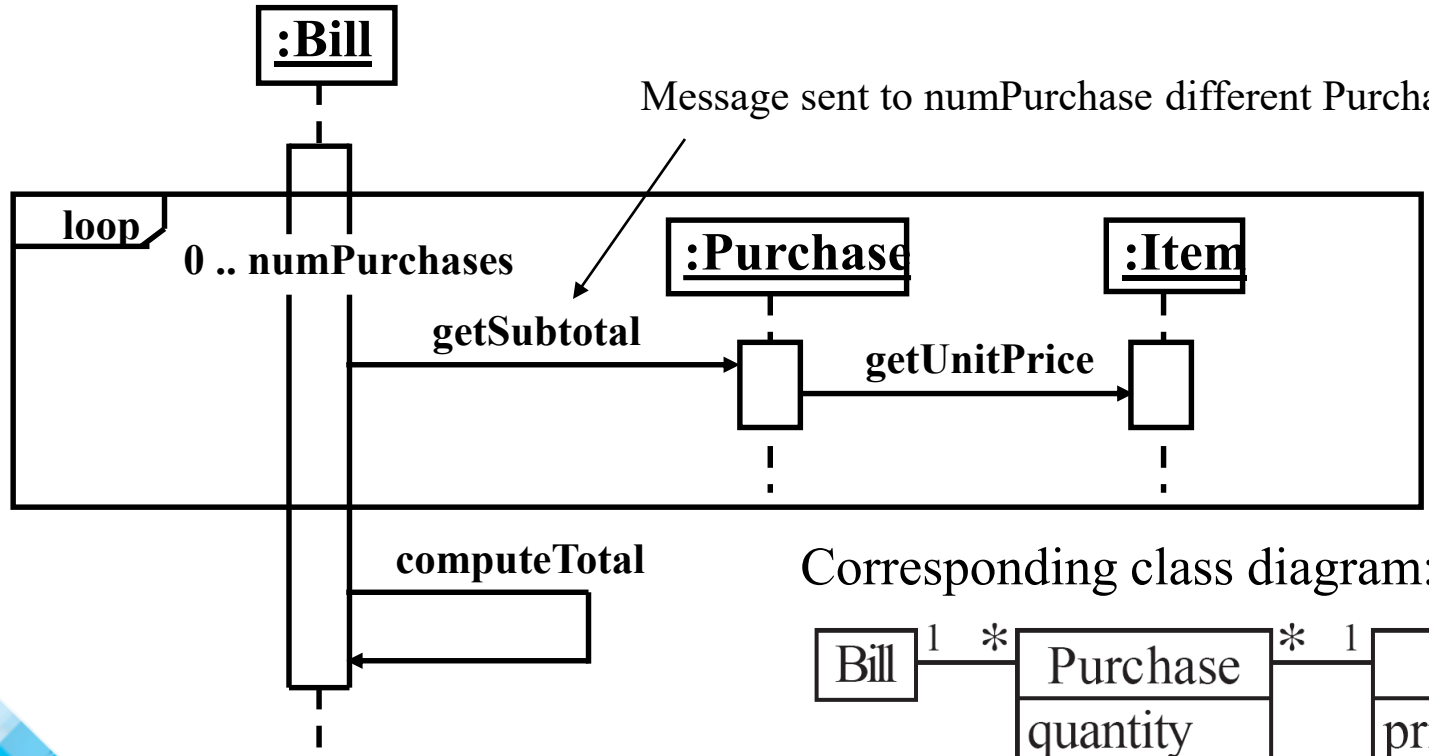
This is a return action, indicating the moment when the computation (the execution of requestToRegister) ends. A return value can (optionally) be shown as a label on a return action.

This is a **combined fragment** (a UML 2.0 feature)

- It shows a subsequence of an interaction.
- 'opt' is a label indicating that it may or may not occur.
- [hasPrerequisite] is a Boolean condition which describes the circumstances when it will occur.

Sequence diagrams – an example with replicated messages

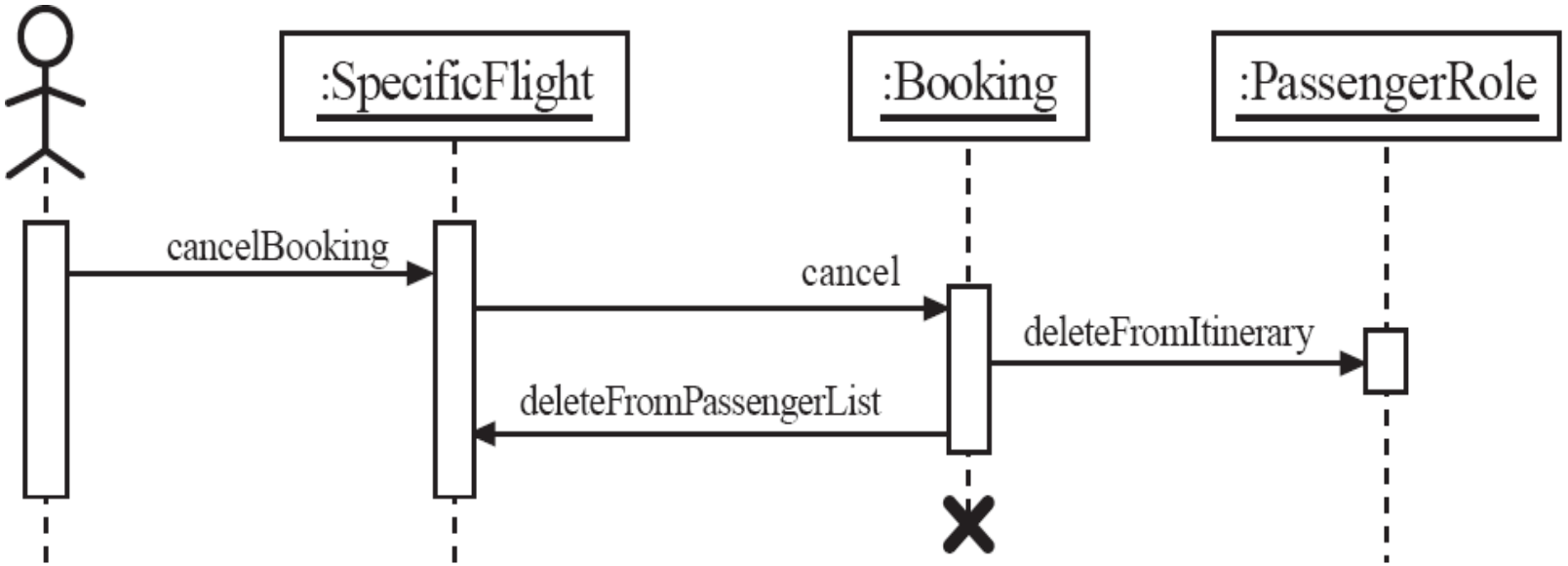
- In a sequence diagram you can show *iteration* using a combined fragment with label 'loop'.
- The number of iterations is specified using the syntax: min .. max.



Corresponding class diagram:

Sequence diagrams – an example with object deletion

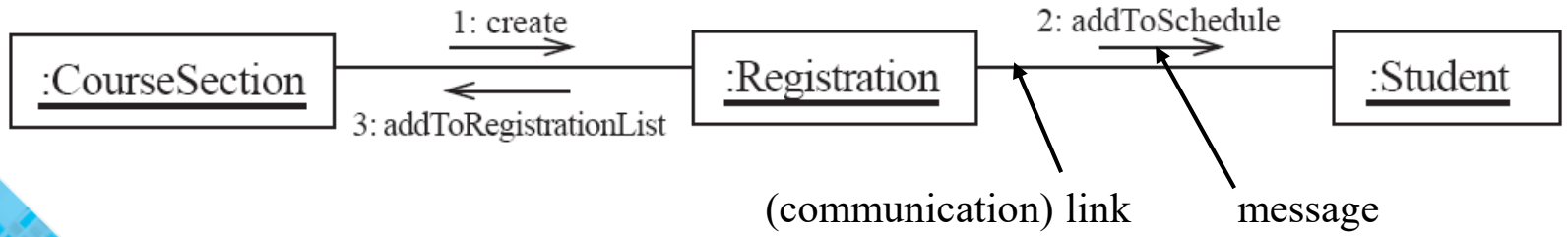
- If an object's life ends, this is shown with an **X** at the end of the lifeline
- This diagram describes a possible interaction for cancelling a Booking in the airline system introduced in chapter 5.



Communication diagrams

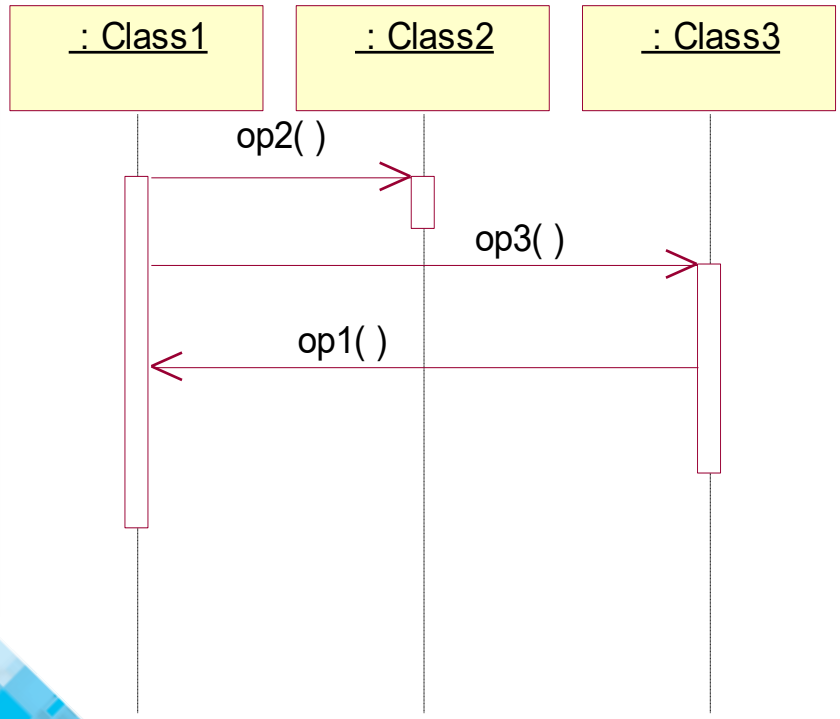
- A *communication diagram* (known as a *collaboration diagram* in UML 1.x) is an interaction diagram that:
  - shows the objects in two dimensions (a graph with objects as vertices)
  - uses sequence numbers to show the order of messages

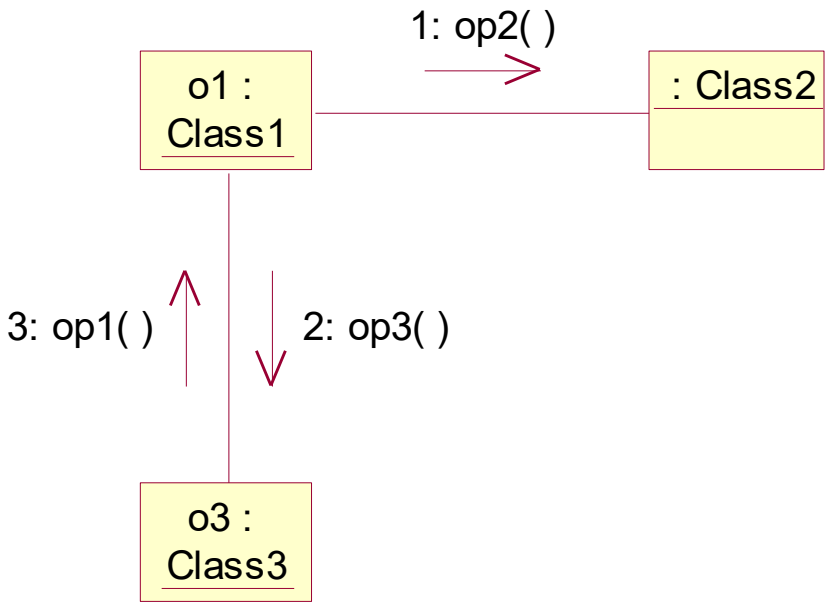
- Communication diagrams and sequence diagrams
  - use different layouts,
  - but are very similar (in UML 1.x they are semantically equivalent).
- Communication diagrams are a bit less expressive than sequence diagrams.
  - Combined fragments - introduced in UML 2.0 (2004) - are lacking in communication diagrams.



Communication and sequence diagrams

Remark: In Rational Rose 2002 (which is based on UML 1.x), you can press F5 to switch between a communication diagram and a corresponding sequence diagram; the two representations of the interaction are isomorphic.





Communication links

- A communication link (in a communication diagram) can exist between two objects whenever it is possible for one object to send a message to the other one.
- The classes of the two interacting objects can be connected by:

  —an association, or

  —a dependency.

8.2 State Diagrams

A state diagram describes the behaviour of a (*sub*)*system*, or an *individual object*.

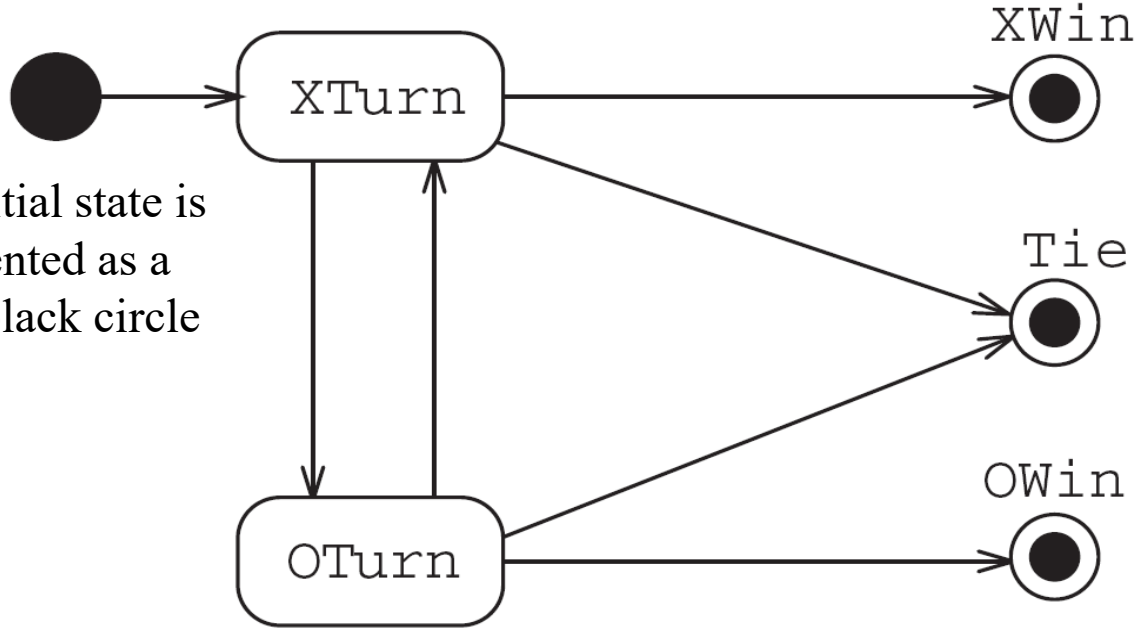
- At any given point in time, the system or object is in a certain state.
  - —Being in a state means that it will behave in a *specific way* in response to any events that occur.
- Some events will cause the system to change state.
  - —In the new state, the system will behave in a different way to events.
- A state diagram is a directed graph where the nodes are states and the arcs are (labelled) transitions.
- A transition represents a change of state in response to an event, and is considered to occur instantaneously (that is, to take no time).
  - —The label of a transition represents the event that causes the change of state.

State diagrams – a first example

- State diagram of a tic-tac-toe (noughts and crosses) game:

A state is rendered as a rectangle with rounded corners containing the name of the state

A transition is rendered as a (solid) arrow connecting two states (in this first example all transitions are unlabelled)



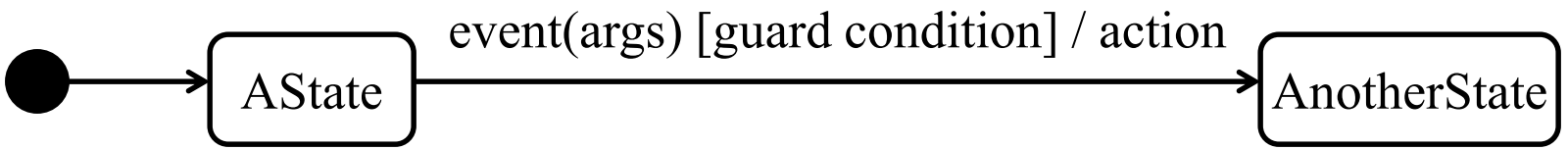
The initial state is represented as a filled black circle

A final state is represented as a filled black circle surrounded by an unfilled circle

Overview of state diagram

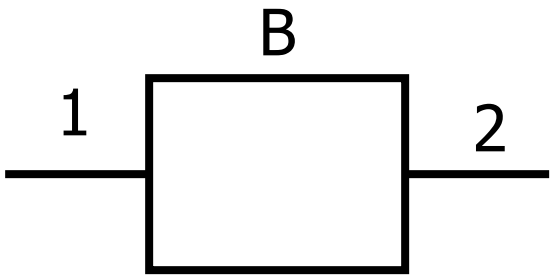
In general, a State Transition Diagram (STD) includes:

- states
- transitions; the specification of a transition can (optionally) include:
  - an event that causes the transition (with optional arguments),
  - a guard condition given between square brackets (if there is more than one transition out of a state there must be mutually exclusive guard conditions on each transition), and
  - an action, which is a non-interruptible behavior that occurs as part of a transition.

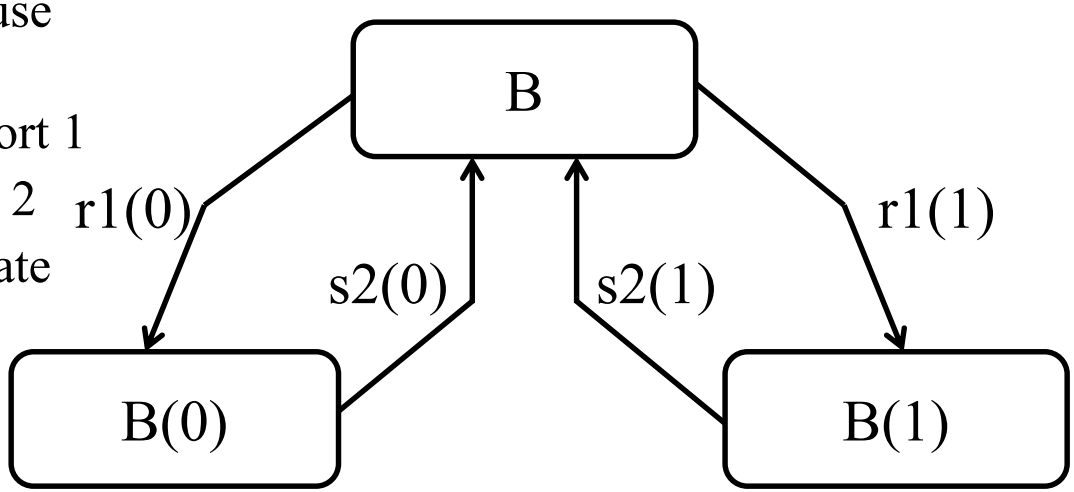


Example: STD specification of a one-bit buffer

- A one-bit buffer is a (software or hardware) component which may hold a single bit.
  - A one-bit buffer B may accept a bit at the input port (1); when holding a bit it may deliver it at the output port (2).

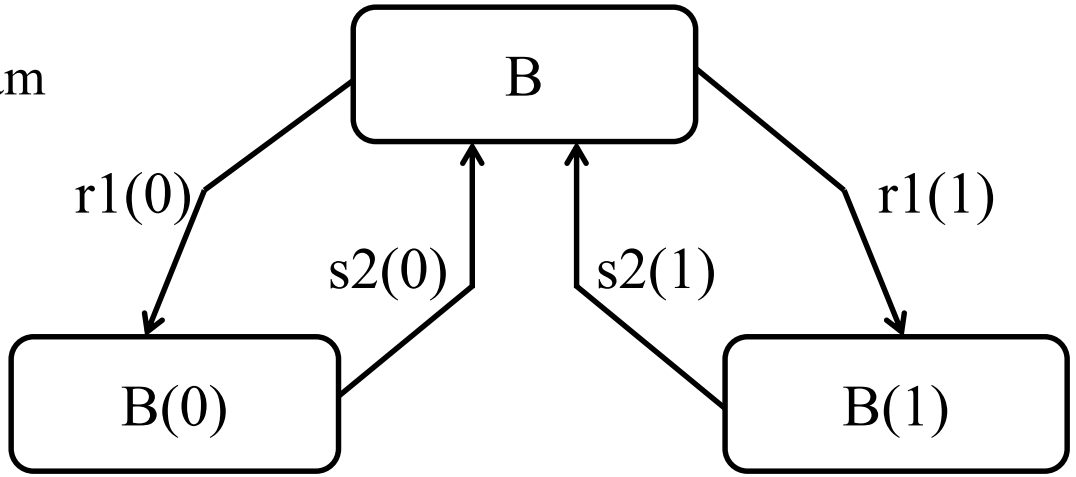


- In the STD specification we use the following conventions:
  - r1(b) = receive bit b at port 1
  - s2(b) = send bit b at port 2
- The STD describes a finite state machine.



State diagrams and formal specification

- A State Transition Diagram (STD) gives an accurate description of the system's behavior.



- The behavior of the one-bit buffer can be specified by the following process algebra equation [BW90]:

$$B = r1(0) . s2(0) . B + r1(1) . s2(1) . B$$

Here

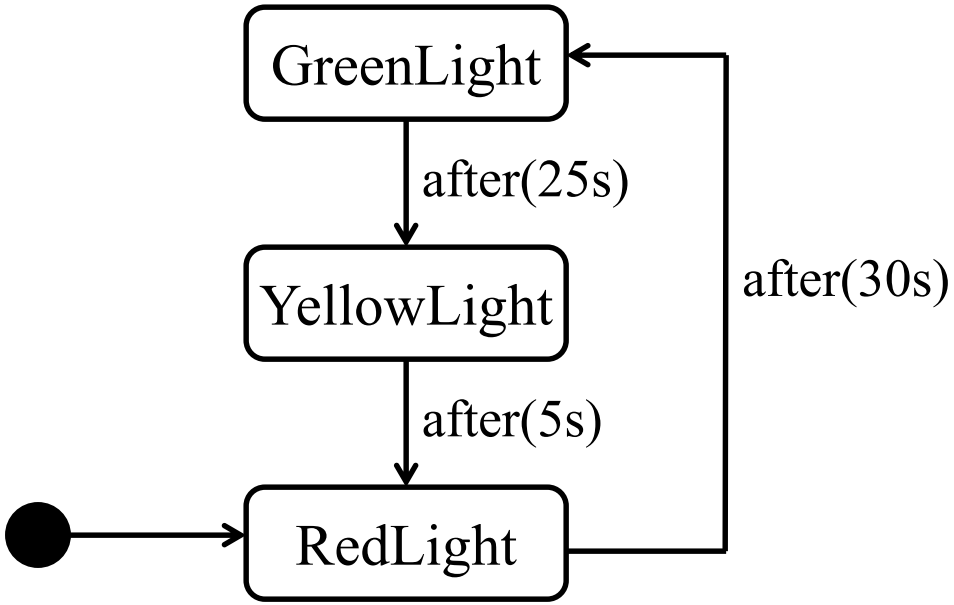
- '.' is the operator for sequential composition
- '+' is the operator for non-deterministic choice.

- In case not just bits, but more generally, elements of some finite data set $(d \in)D$ are buffered we get the following equation:

$$B = \sum_{d \in D} r1(d) . s2(d) . B$$

State diagrams with elapsed-time transitions

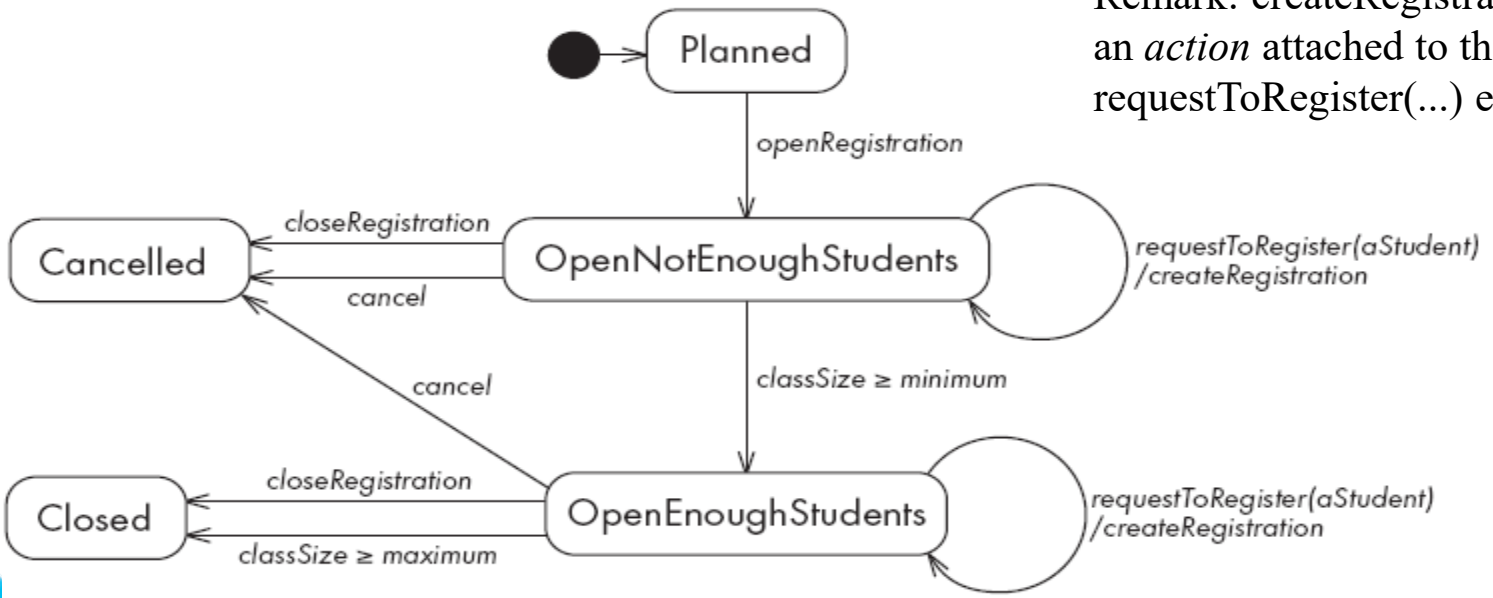
- The event that triggers a transition can be a certain amount of elapsed time.
- The STD given below illustrates the use of elapsed-time transitions to model the behavior of a simple traffic light.



State diagrams – an example with conditional transitions

- This STD describes the behavior of instances of the CourseSection class.
  - It includes two transitions triggered by conditions becoming true.
  - For example, in the OpenNotEnoughStudents state a CourseSection object makes a transition to the OpenEnoughStudents state (in which the course can actually be taught) as soon as the (classSize >= minimum) condition becomes true.

Remark: createRegistration is an *action* attached to the requestToRegister(...) event.



Activities in state diagrams

- An *activity* is something that takes place while the system is *in* a state.

  —It takes a period of time.

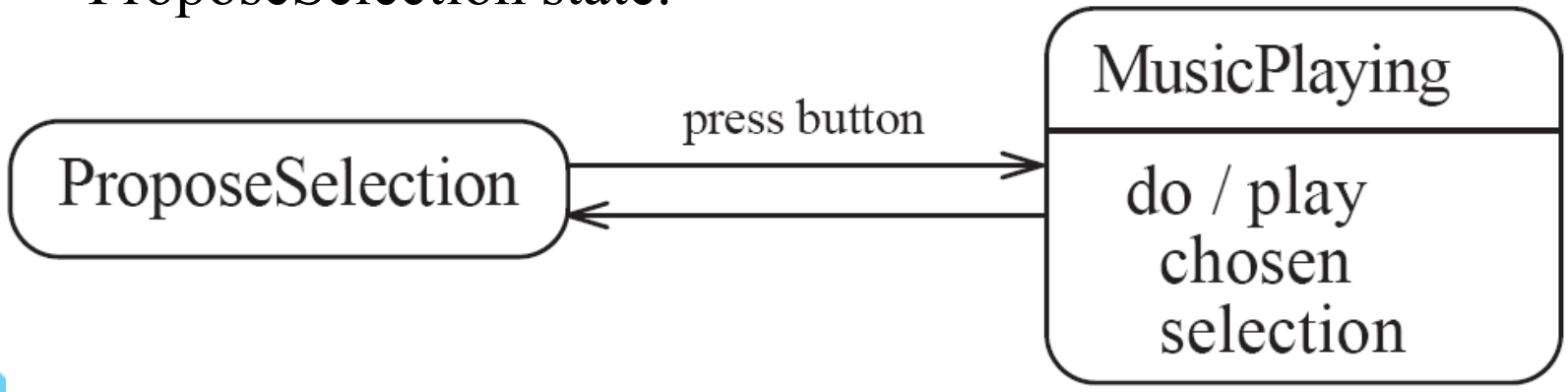
  —An activity is shown inside the state itself, preceded by the string “do /” .

  —The system takes a transition out of the state in response to completion of the activity (if no other transition is triggered before by some event).

State diagram – an example with activity

State diagram for a jukebox illustrating an activity in a state.

- In the ProposeSelection state the system waits for the user to press a button, selecting some music.
- In the MusicPlay state the system plays the chosen music until it comes to an end.
    - The system then takes a transition back to the ProposeSelection state.

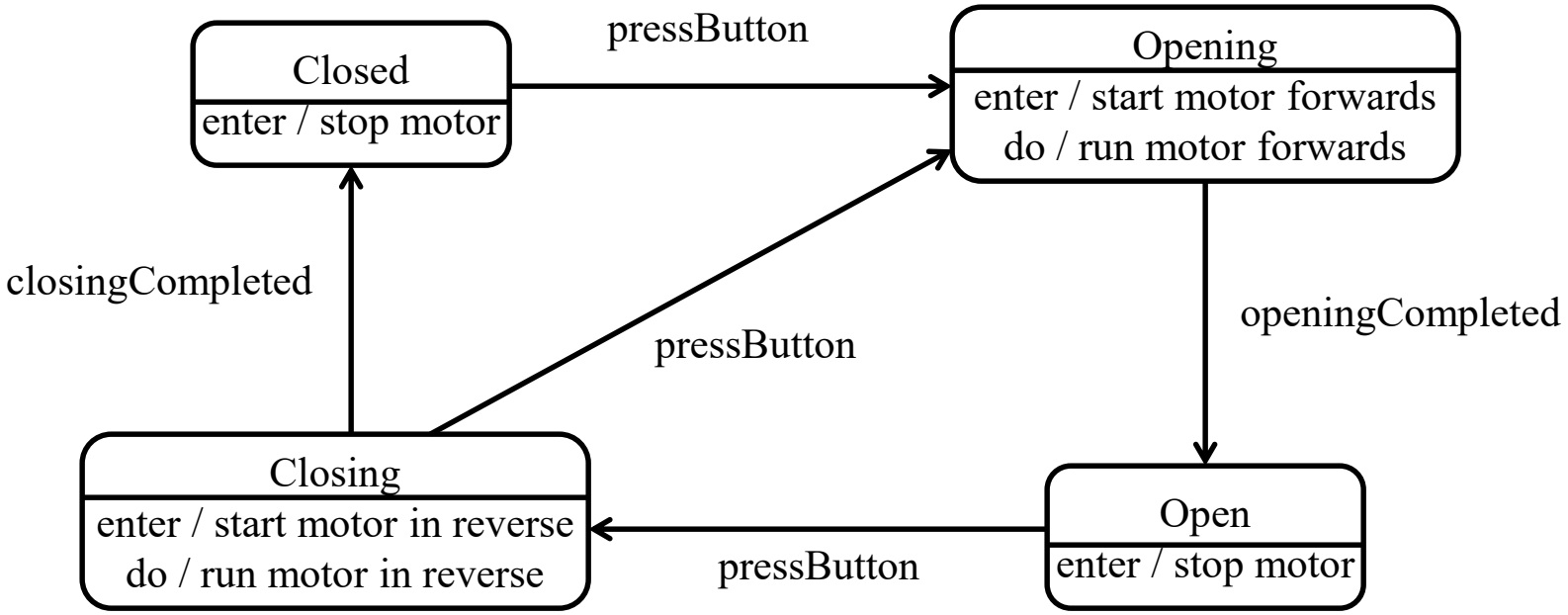


Actions in state diagrams

- An *action* is something that takes place effectively *instantaneously*
  - —When a particular transition is taken (in this case the action is attached to an event)
  - —Upon entry into a particular state
    - An entry action is shown inside the state, preceded by the string “entry /”
  - —Upon exit from a particular state
    - An exit action is shown inside the state, preceded by the string “exit /”
- An action should consume no noticeable amount of time

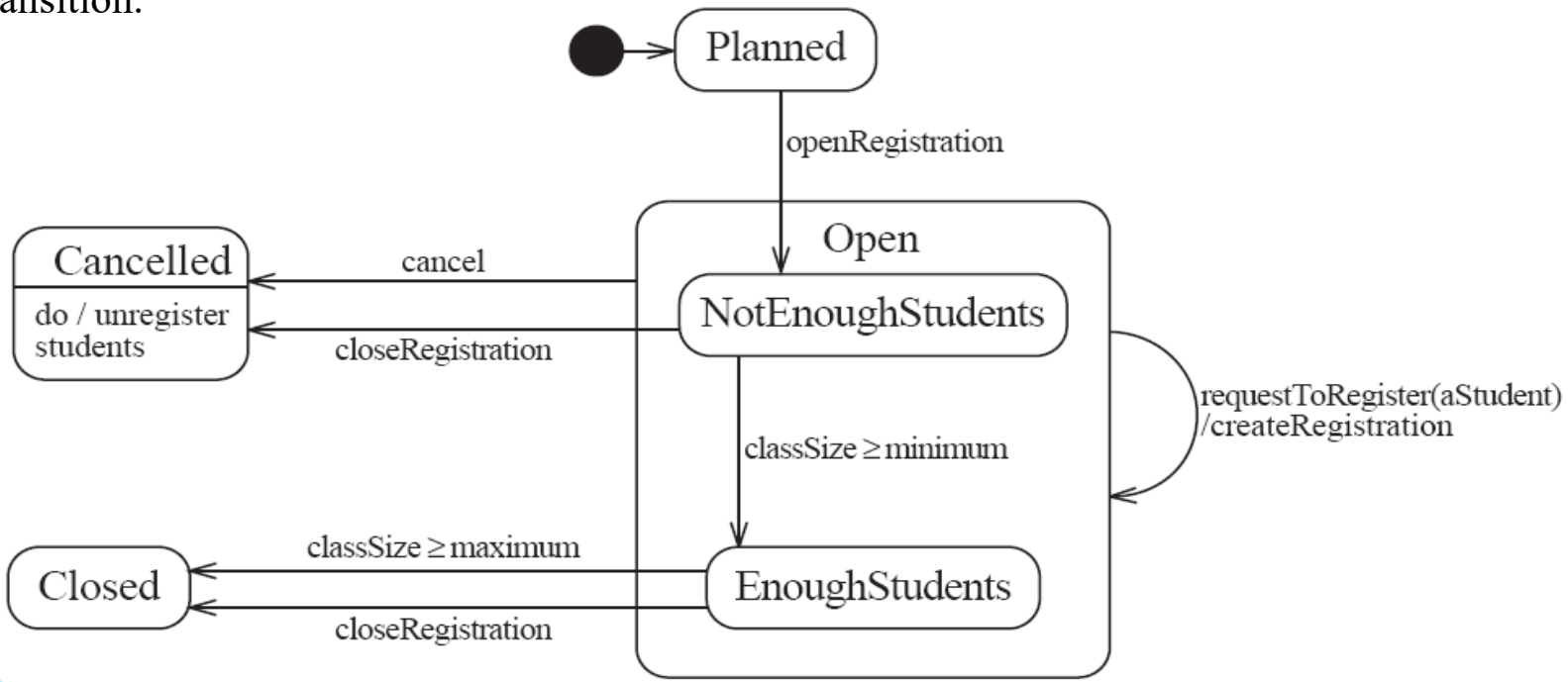
State diagram – an example with actions

State diagram for a garage door opener



State diagram – an example with substates

- A state diagram can be nested inside a state. The states of the inner diagram are called *substates*.
  - If two or more states have an identical transition, they can be grouped together in a superstate. Then, rather than maintaining two identical transitions (one for each substate) the transition can be moved to the superstate.
- The figure shows the behavior of instances of the CourseSection class, converted to use nested substates. It is now enough to show only one 'cancel' transition and one 'requestToRegister' transition.

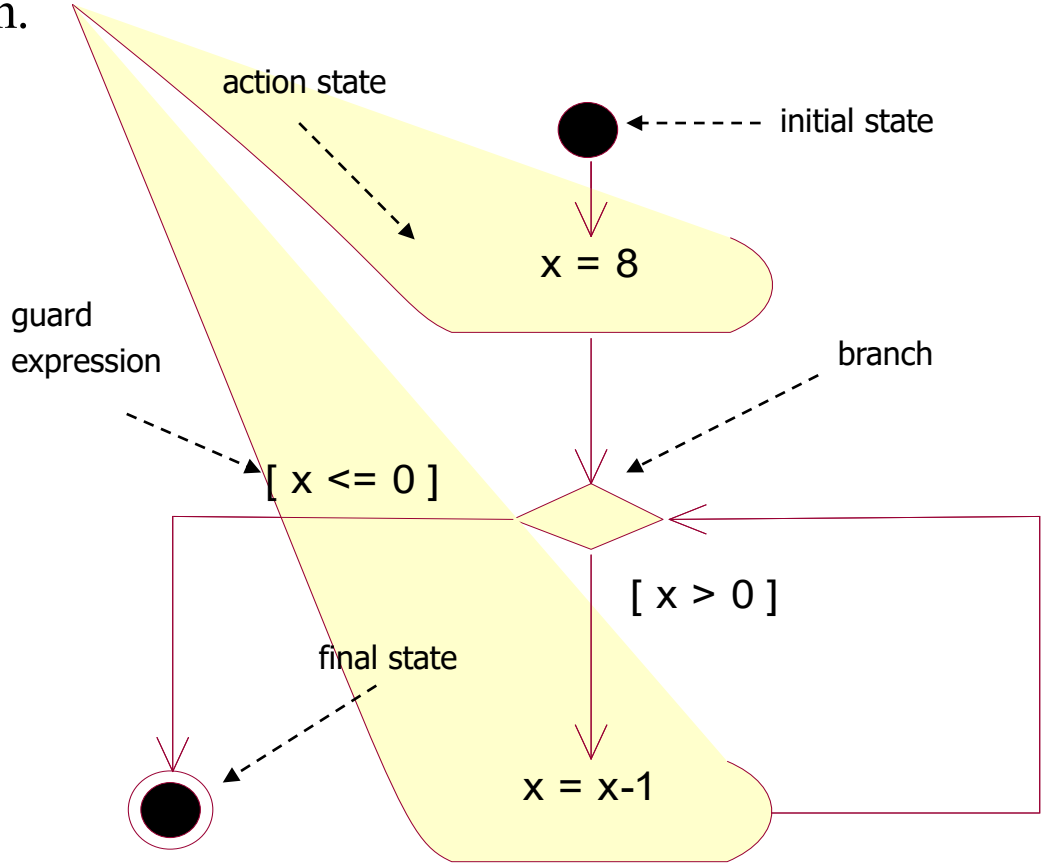


8.3. Activity diagrams

- An activity diagram is used to understand the flow of work that an object or a system performs.
- An *activity diagram* is a special kind of a state transition diagram, that shows the flow from activity to activity within a system. In an activity diagram the states are activity states and the flow of control passes immediately to the next state when the activity (of the current state) completes [BRJ99].
  - An *activity state* represents the performance of a non-atomic task within a state machine.
  - An *action state* is a special kind of an activity state that cannot be further decomposed (the internal activity of an action state is a single atomic computation).

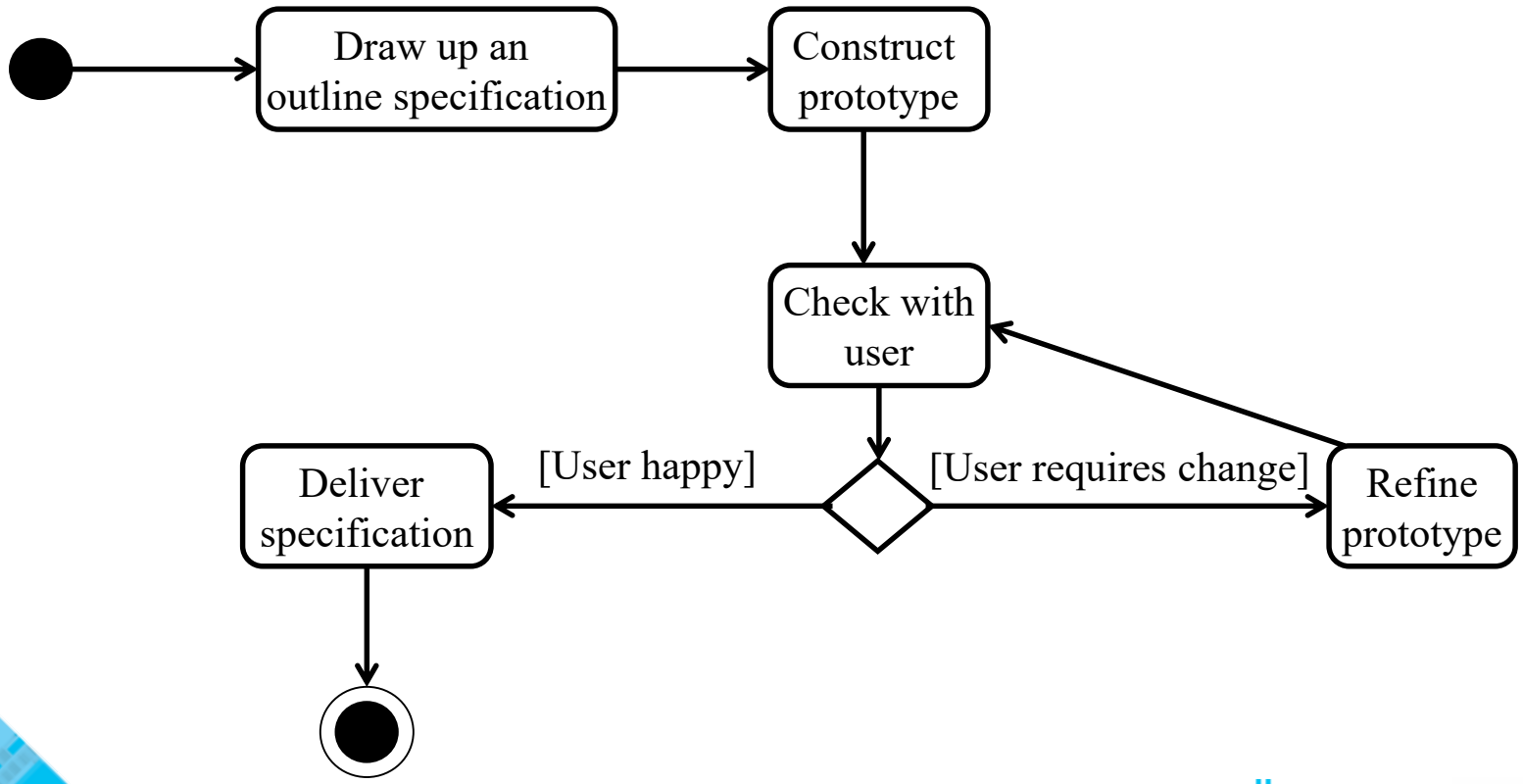
Activity diagram – an example with action states

An activity diagram can use action states to show the flowchart of an imperative program.



Activity diagram – an example with activity states

- An activity diagram can use activity states to show human activities, such as software processes.
- The figure shows an activity diagram for throwaway prototyping [Bel05]:



Additional references

[BW90] J.C. Baeten, W.P. Weijland. *Process algebra*. Cambridge University Press, 1990.

[Bel05] D. Bell. *Software Engineering for Students – a Programming Approach* (4th edition). Addison-Wesley, 2005.

[BRJ99] G. Booch, J.Rumbaugh, I. Jacobson. *The Unified Modeling Language User Guide*. Addison Wesley, 1999.